

Conceitos errôneos de programação para estudantes

Alaaeddin Suécia, Felienne Hermans e Marileen Smith

Universidade de Tecnologia de Delft
Delft, Países Baixos

{alaaeddin.swidan,ffjhermans,miesmit}@tudelft.nl

RESUMO

Conceitos errôneos sobre programação têm sido um tema de interesse no ensino introdutório de programação, com foco no nível universitário. alunos de nível. Atualmente, a programação é cada vez mais ensinada a crianças mais novas nas escolas, às vezes como parte do currículo. Neste estudo, pretendemos explorar quais concepções errôneas são mantidas por crianças mais novas, em idade escolar. Para isso, elaboramos um questionário de múltipla escolha com exercícios de programação em Scratch. As perguntas representam um conjunto selecionado de 11 equívocos conhecidos e relacionados a conceitos básicos de programação. 145 participantes com idades entre 7 e 17 anos, com experiência em programação, participaram do estudo. Nossos resultados mostram que os três principais equívocos comuns são: **o desempenho dos alunos. O efeito começa cedo [24] e pode permanecer por um longo tempo [17]. Foi constatado que eles causam falhas em cursos introdutórios de programação e, a longo prazo, até mesmo fazer com que os alunos abandonem a educação em programação [12]. Estudos anteriores, no entanto, concentraram-se em cursos introdutórios em universidades. Atualmente, o ensino de Ciência da Computação e a programação estão sendo cada vez mais introduzidos a alunos mais jovens no ensino fundamental e médio. escolas [2, 10]. Muitos países já integraram atividades de programação em seus currículos escolares [9]. Além disso, novas Linguagens de programação e ambientes de programação são implementados especialmente para crianças mais novas. Um exemplo é o Scratch1. uma linguagem baseada em blocos desenvolvida pelo MIT com o objetivo de ensinar crianças a programar. Embora a educação em ciência da computação esteja chegando às escolas, pouco se sabe sobre se as crianças desenvolvem Nesta fase, existem algumas concepções errôneas sobre programação. Este estudo tem como objetivo explorar essas concepções errôneas que crianças em idade escolar possuem. Desenvolvemos um questionário de múltipla escolha contendo 11 perguntas que representam um conjunto selecionado de equívocos sobre programação. conhecido de pesquisas anteriores [21]. No total, 145 crianças com idades entre 7 e 17 anos participaram do nosso estudo. Os participantes, que Além disso, era necessário ter experiência em programação. Eles apresentaram justificativas para suas respostas em textos abertos. De Com os dados coletados nesta pesquisa, pretendemos responder às seguintes perguntas: Questões de pesquisa:**

Uma variável armazena um valor por vez, e isso influencia a interatividade de um programa. Quando a entrada do usuário é necessária. Ter uma ideia errada influencia pelo efeito matemático dos números, significado semântico dos identificadores e altas expectativas sobre o que um computador pode fazer. Outros Os resultados mostram que as crianças mais velhas respondem mais vezes.

As perguntas foram respondidas corretamente, especialmente no que diz respeito aos conceitos de variáveis e controle. Crianças que programam apenas em Scratch parecem ter dificuldades. ao responderem corretamente às perguntas em comparação com crianças que programa em Scratch e outra linguagem. Nossos resultados sugerem que O trabalho deve se concentrar na identificação de equívocos induzidos pelo Scratch. e desenvolver métodos de intervenção para combater essas ideias errôneas. o mais cedo possível. Por fim, para crianças que começam a aprender programação com o Scratch, os materiais devem ser mais ricos em conceitos e Incluir diversos exercícios para cada conceito.

Formato de referência da ACM:

Alaaeddin Swidan, Felienne Hermans e Marileen Smit. 2018. Conceitos errôneos sobre programação para estudantes. In ICER '18: Conferência Internacional de 2018 Conferência de Pesquisa em Educação Computacional, 13 a 15 de agosto de 2018, Espoo, Finlândia . ACM, Nova York, NY, EUA, 9 páginas. <https://doi.org/10.1145/3230977.3230995>

1 INTRODUÇÃO

É sabido, através de pesquisas existentes, que aprender a programar é difícil [3, 4, 11]. Uma fonte de dificuldades é manter concepções errôneas de programação [4, 24], o que afeta o desempenho na escrita ou entender código. Um equívoco comum na programação é ter

É concedida permissão para fazer cópias digitais ou impressas, totais ou parciais, desta obra para uso pessoal ou O uso em sala de aula é concedido gratuitamente, desde que não sejam feitas ou distribuídas cópias. para fins lucrativos ou comerciais, e que as cópias contenham este aviso e a citação completa. na primeira página. Os direitos autorais dos componentes desta obra pertencem a terceiros. Os autores devem ser respeitados. É permitido fazer resumos com a devida atribuição. Para copiar de outra forma, ou A republicação, seja para postar em servidores ou redistribuir para listas, requer autorização prévia específica. e/ou uma taxa. Solicite as permissões através do endereço permissions@acm.org.

ICER '18, 13 a 15 de agosto de 2018, Espoo, Finlândia

© 2018 Direitos autorais pertencem ao(s) proprietário(s)/autor(es). Direitos de publicação licenciados para o Associação para Maquinaria de Computação.

ACM ISBN 978-1-4503-5628-2/18/08... \$15.00

<https://doi.org/10.1145/3230977.3230995>

uma compreensão incorreta de um conceito de programação ou de um conjunto de conceitos relacionados, geralmente afetados por conhecimento prévio de domínios diferentes da programação, como matemática e ciências naturais. línguas [20].

Estudar equívocos comuns em programação envolve identificar seus componentes. possíveis origens para que tanto alunos quanto educadores possam retificar conceitos relacionados. Conceitos errôneos têm um efeito prejudicial sobre o desempenho dos alunos. O efeito começa cedo [24] e pode permanecer por um longo tempo [17]. Foi constatado que eles causam falhas em cursos introdutórios de programação e, a longo prazo, até mesmo fazer com que os alunos abandonem a educação em programação [12]. Estudos anteriores, no entanto, concentraram-se em cursos introdutórios em universidades. Atualmente, o ensino de Ciência da Computação e a programação estão sendo cada vez mais introduzidos a alunos mais jovens no ensino fundamental e médio. escolas [2, 10]. Muitos países já integraram atividades de programação em seus currículos escolares [9]. Além disso, novas Linguagens de programação e ambientes de programação são implementados especialmente para crianças mais novas. Um exemplo é o Scratch1. uma linguagem baseada em blocos desenvolvida pelo MIT com o objetivo de ensinar crianças a programar. Embora a educação em ciência da computação esteja chegando às escolas, pouco se sabe sobre se as crianças desenvolvem Nesta fase, existem algumas concepções errôneas sobre programação. Este estudo tem como objetivo explorar essas concepções errôneas que crianças em idade escolar possuem. Desenvolvemos um questionário de múltipla escolha contendo 11 perguntas que representam um conjunto selecionado de equívocos sobre programação. conhecido de pesquisas anteriores [21]. No total, 145 crianças com idades entre 7 e 17 anos participaram do nosso estudo. Os participantes, que Além disso, era necessário ter experiência em programação. Eles apresentaram justificativas para suas respostas em textos abertos. De Com os dados coletados nesta pesquisa, pretendemos responder às seguintes perguntas: Questões de pesquisa:

RQ1 Quais são os equívocos mais comuns entre os programadores iniciantes em Scratch em relação à programação?

RQ2 Como as crianças que têm essas ideias erradas explicam isso? Suas respostas? Em que diferem suas explicações das Será que alguma criança está entendendo o conceito corretamente?

RQ3 Como a idade e o conhecimento prévio de programação afetam Ter uma ideia errada ou um entendimento correto?

2. CONTEXTO

A pesquisa define um equívoco em programação como uma compreensão incorreta de um conceito ou conjunto de conceitos, o que leva a erros. erros na escrita ou leitura de programas [20]. Conceitos errôneos podem estar relacionado a conceitos básicos, porém fundamentais, de programação, não apenas para conceitos avançados. Além de erros sintáticos, há

1<https://scratch.mit.edu/>

parece haver um consenso entre os pesquisadores sobre determinados conceitos que são difíceis para os aprendizes. Esses conceitos independentes de linguagem incluem variáveis, loops e declarações condicionais [4, 7, 11, 13, 17]. Um exemplo de equívoco comum na programação: a crença de que uma variável pode assumir múltiplos valores, ou a crença de que uma variável A atribuição segue na direção oposta [4, 21]. Na orientação a objetos linguagens, dificuldades comuns estão relacionadas ao escopo e visibilidade de variáveis, modularização e decomposição e herança [8, 12].

A pesquisa se concentrou em entender a origem de uma ideia equivocada. Uma ideia equivocada sobre programação não significa que

O aluno demonstra total desconhecimento, ou seja, isso indica que ele não possui nenhum conhecimento prévio. Conhecimento parcial, mas autointerpretado, que provém de domínios além da programação [4]. Algumas das origens conhecidas incluem o uso de analogias específicas para explicar um conceito, os significados ambíguos e ambíguos de algumas palavras-chave de programação em inglês como língua natural e matemática [4, 15, 19]. Du

Boulay [4] introduziu o que chamou de “máquina conceitual” como Uma das origens dos equívocos em programação. A máquina conceitual. Refere-se às propriedades gerais que um aluno assume da máquina que executa seu código. Ter uma compreensão incorreta de Acredita-se que a máquina conceitual de uma linguagem de programação seja ser a causa de muitos equívocos [13, 20]. Por exemplo, erros foram encontradas como resultado do que Pea [14] chamou de “superbactéria”, que é a suposição de que “existe uma mente oculta em algum lugar no linguagem de programação que possui poderes inteligentes e interpretativos” [14, p. 32], ou “esquecer-se de ramos alternativos porque são muito óbvio merecer consideração” [22, p. 6].

Finalmente, a variedade de concepções errôneas dificulta que os educadores as levem totalmente em consideração. A este respeito, Sorva [21] fornece uma lista abrangente de equívocos comuns sobre programação, compilada a partir de vários estudos [4, 6, 12, 19, 20]. Neste estudo, utilizamos A lista de Sorva como ponto de partida para investigar equívocos sobre o Scratch em estudantes.

3 CONFIGURAÇÃO

O objetivo do nosso estudo é explorar os equívocos mais comuns, entre crianças em idade escolar no Scratch. Para isso, realizamos um Estudo baseado em questionário. Os participantes recebem um conjunto de perguntas de múltipla escolha; cada pergunta consiste em um exercício de programação em O Scratch testa a aceitação de uma ideia equivocada conhecida. Questionários têm sido usados para avaliar a aceitação de conceitos errôneos em programação. concepções errôneas em pesquisas anteriores [12, 18]. Os participantes na maioria Os casos exigem que se preveja o resultado do roteiro para escolher uma resposta. A Figura 2 mostra um exemplo de pergunta do nosso estudo. Fornecemos o Questionário completo (versão em inglês) aqui2. Nas seções seguintes Descrevemos detalhadamente a configuração do estudo.

3.1 Participantes

No total, 145 crianças participaram do estudo. A Figura 1 mostra a distribuição dos participantes de acordo com a idade relatada, variando entre 7 e 17 anos. Entre os participantes, 102 (70,3%) são meninos, e 51 (29%) são meninas; um participante não especificou o gênero.

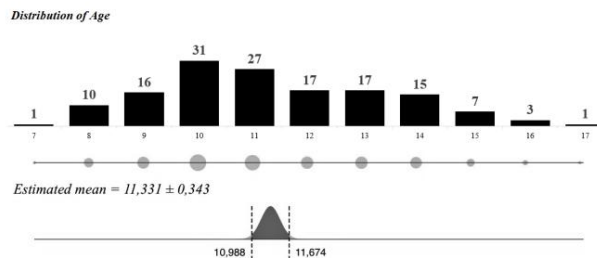


Figura 1: Distribuição etária dos participantes. 70,3% deles são alunos do ensino fundamental (com 12 anos de idade ou menos)

3.2 Ambiente de estudo

Realizamos este experimento no museu de ciências NEMO. As 3 Em Amsterdã. crianças que visitavam o museu foram convidadas a participar do experimento. sobre programação, mas não recebi mais informações sobre o que era. O experimento mediria os resultados. Os participantes não receberam compensação financeira pela participação, mas receberam um certificado. esforços. O experimento teve uma janela de 25 minutos por criança e foi realizada em uma sala separada do museu com capacidade para 8 pessoas. crianças de cada vez. No total, passamos 14 dias no museu, correndo O experimento durou cerca de 5 horas por dia. Durante o experimento Cada participante teve que preencher as respostas ao nosso questionário online. em que as perguntas foram apresentadas em imagens estáticas. As crianças tinham acesso à máquina e à Internet, mas não observamos Nenhum participante abriu outros aplicativos ou páginas além do questionário. O número de participantes relatado neste estudo é um Para filtrar o subconjunto total de crianças que visitaram nosso estande, selecionamos... participantes que não tinham experiência em programação ou que indicaram que estavam adivinhando as respostas. Fazer perguntas às crianças nesse contexto nos permitiu estudar a prevalência de concepções errôneas em uma amostra que é menos dependente de metodologias de ensino específicas em comparação com Um experimento realizado em uma escola.

3.3 Seleção de equívocos

Sorva fornece uma lista abrangente de 162 concepções errôneas de programação conhecidas na literatura [21]. Usamos a lista como nosso Ponto de partida para investigar equívocos sobre o Scratch. Seguimos uma abordagem em duas etapas para alcançar esse objetivo. **Primeiro, selecionamos o mais Equívocos comuns da lista.** Um equívoco é comum se indicado por pelo menos duas pesquisas distintas. Esta etapa Reduzimos a lista para 17 equívocos comuns sobre programação. **Em segundo lugar, visto que Ao estudarmos equívocos sobre o Scratch, filtramos aqueles que não se encaixam no Scratch como linguagem.** Por exemplo, nós Eliminamos duas ideias equivocadas sobre o uso de uma variável de controle de loop dentro do corpo do loop. No Scratch, os loops usam uma variável estática. O valor e nenhuma variável são necessários para iterar pelo corpo. Isso resultaram em 11 equívocos, apresentados na Tabela 1.

3.4 Perguntas

Antes de apresentar as questões sobre equívocos aos participantes, Solicitamos que o participante responda a cinco perguntas fechadas. no questionário. As respostas a essas perguntas indicam o

²<http://cli.re/g1zyQM>

³<https://www.nemosciencemuseum.nl/en/>

Tabela 1: Conceitos errôneos de programação incluídos em nosso estudo de [21].

Descrição do Código 4	Pré-requisito Conceito(s)5	Descrição da pergunta (Pseudocódigo)	Escolha(s) de Conceito Errado
M9 Uma variável pode armazenar múltiplos valores simultaneamente. A atribuição primitiva M11 funciona na direção oposta às variáveis.	Variáveis	Defina [X] como 10; Defina [X] como 20; Diga [X]; Defina [a] como 10; Defina [b] como 20; Defina	Gobo diz 10, 20 a=20, b=0 a=20, b=10
M14 Uma variável é uma associação de um nome a um valor que pode ser alterado. Ela não é armazenada dentro do computador.	Variáveis	[a] como [b]; Criamos uma variável chamada mensagem. Onde a variável é armazenada?	A variável não está armazenada em nenhum lugar. A variável só é visível na tela.
M15 Atribuição primitiva armazena equações ou expressões não resolvidas.	Variáveis	Defina [X] como 1, defina [contador] como [X+1]; Diga [contador];	Gobo diz X+1
M17 A semântica da linguagem natural dos nomes de variáveis afeta qual valor é atribuído a qual variável.	Variáveis	Defina [grande] como 1; Defina [pequeno] como 100; Defina [grande] para [pequeno];	grande=100 pequeno=1 grande=100 pequeno=0
M23 Dificuldades na compreensão da sequencialidade das declarações	Variáveis	Defina [número1] como 0; Defina [número2] como 0; Defina [total] como [número1] + [número2]; Defina [número1] como 4; Defina [número2] como 2; Diga [total];	Gobo diz 6
M26 Uma condição falsa encerra o programa se não houver um ramo else.	SE SENÃO	SE tocar na cor [preto] então { Diga "Ai!!"; }; Diga "Estou me movendo"; Defina [contador] para 0; Repita 5 vezes { Altere [contador] em 1; }; Diga [contador];	Gobo não diz nada.
M30 O código adjacente é executado dentro do loop	Variáveis e Laços	Diga "Estou me movendo"; SE tocar na cor [preto] então { Diga "Ai!!"; }; Diga "Pronto!!"; Defina [número] para 1; Repita até [número]=3 { Altere [número] em 1; Diga [número]; };	Gobo diz de 1 a 5
M31 O controle retorna ao início quando a condição for falsa SE SENÃO	Variáveis e Laços	Pergunte ["Quantos anos você tem?"] e espere; diga "Legal! Vou me mexer agora"; mova-se [10] passos;	Gobo diz: "Estou me mudando".
Os loops M33 terminam assim que a condição muda para falso	Nenhum		Gobo diz 2
M150 Dificuldades em compreender o efeito das chamadas de entrada na execução			Gobo pergunta: "Quantos anos você tem?" e imediatamente responde: "Legal! Vou me mexer agora" e dá 10 passos.

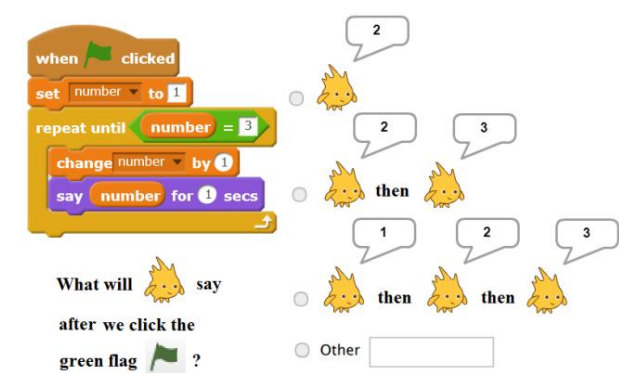


Figura 2: Pergunta e possíveis respostas para M33

A familiaridade do participante com estes conceitos de programação: variáveis, instruções IF e loops. Usamos as respostas em uma lógica de ramificação automática para apresentar um conjunto de três a onze perguntas de múltipla escolha sobre concepções errôneas (ver Tabela 1) que se adequam ao conhecimento do participante. Cada pergunta é elaborada para suscitar uma das seguintes questões

concepções errôneas de programação bem conhecidas. Para isso, utilizamos problemas de programação semelhantes aos sugeridos por Ma [12] para estudantes de Java. Em nosso caso, elaboramos a questão no Scratch, tanto em inglês quanto em holandês. O Scratch permite a programação no idioma nativo, o que elimina a carga cognitiva da leitura em língua estrangeira para as crianças locais e permite que elas se concentrem no desafio de programação [5, 24]. A Figura 2 mostra um exemplo de uma das questões relacionadas à concepção errônea de que um loop termina assim que a condição se torna falsa (M33). Pedimos aos participantes que prevejam o resultado do programa perguntando “O que o Gobo dirá quando clicarmos na bandeira verde?”, onde Gobo é um dos personagens conhecidos do Scratch. As respostas são categorizadas em “Resposta Correta” (Gobo diz 2 e depois 3), “Resposta Errada”

(Gobo diz 2) e Outra_Errada (Gobo diz 1, depois 2 e depois 3). Segue-se uma pergunta aberta para que o participante possa explicar os motivos da resposta escolhida. Usamos o texto da pergunta aberta para filtrar os resultados. O objetivo dessa filtragem é eliminar respostas incorretas.

4Adicionamos um prefixo, M, aos números originais atribuídos à concepção errônea na lista de Sorva. Isso visa facilitar a referência no artigo.
5Esta coluna indica como atribuímos perguntas aos participantes com base nos conceitos errôneos que eles relatam.

Para as quais as crianças admitiram que ou chutaram a resposta ou não a entenderam. Observamos que inicialmente recebemos 1.306 respostas de 178 participantes. Devido ao processo de filtragem, 545 respostas foram eliminadas. O número de participantes incluídos no estudo foi reduzido para 145, visto que, para alguns participantes, todo o conjunto de respostas foi eliminado.

4 RESULTADOS

Esta seção apresenta uma visão geral das respostas às questões de pesquisa do estudo, com base nos dados do questionário.

4.1 Conceitos errôneos mais comuns [RQ1] Quais são

os conceitos errôneos de programação mais comuns entre os programadores iniciantes em Scratch?

Para responder à RQ1, analisamos as respostas para cada questão sobre concepções errôneas. A Figura 3 apresenta a porcentagem de participantes que selecionaram a opção de concepção errônea para cada questão. As três concepções errôneas mais comuns estão relacionadas a conceitos diferentes: (i) a sequencialidade da execução do código (M23), (ii) a variável que pode armazenar múltiplos valores simultaneamente (M9) e (iii) a interação humano-computador e seu efeito na execução (M150). Além disso, observamos que, entre as concepções errôneas menos comuns, estão aquelas relacionadas a estruturas de controle: laços e condicionais (M31, M33 e M26). Nesses casos, porém, opções incorretas são populares entre os participantes, o que indica a dificuldade geral em compreender esses conceitos.

4.2 Percepções a partir das explicações das crianças [RQ2]

Como as crianças que têm essas concepções errôneas explicam suas respostas? Como as explicações delas diferem das explicações das crianças que entendem o conceito corretamente?

Primeiramente, analisamos quantitativamente o texto aberto fornecido pelos participantes, de acordo com a categoria de resposta, para as três concepções errôneas mais comuns (ver Tabela 2). Para destacar o processo de raciocínio das crianças, tanto daquelas que apresentavam uma concepção errônea quanto daquelas que possuíam a concepção correta, exploramos ainda as respostas abertas fornecidas pelos participantes para M14 e M26, além daquelas presentes na Tabela 2. M14 foi escolhido porque dois aspectos da concepção errônea são apresentados nas múltiplas opções. A opção M26 foi escolhida porque, embora uma opção que represente a concepção errônea não seja fornecida separadamente (ver Seção 5.4), os participantes ainda indicaram possuir a concepção errônea por meio do texto aberto fornecido.

M23: Dificuldades na compreensão da sequência de enunciados

Participantes com a concepção errada = 56,2%, Concepção correta = 42,5%, n = 73

Conceito errôneo: Os participantes demonstram foco na operação matemática em si, e não na sequência. Isso fica evidente nas três palavras mais frequentes, que incluem "valores" e "somar". Além disso, encontramos explicações como "porque se você somar o número 1 (4) e o número 2 (2), o resultado será 6" ou simplesmente "Matemática básica, cara". Alguns participantes presumiram um aspecto automático da operação: "quando você altera os valores dessas variáveis, o valor total também muda" e "4 + 2 = 6, o computador deve calcular isso para você".

Correto: Os participantes conseguem identificar a natureza sequencial do código. Nas palavras mais frequentes, encontramos a palavra "antes", que indica uma ordem. Um participante, por exemplo, explica: "Como o total é definido como [nº 1] + [nº 2], ele é igual a 0. As variáveis alteradas depois disso são irrelevantes". Outro participante sugeriu uma "correção" para o código: "Se o bloco que define [total] como [número 1] + [número 2] fosse colocado mais abaixo, teria funcionado".

M9: Uma variável pode armazenar múltiplos valores simultaneamente / "lembra" valores

antigos. Participantes com a concepção errônea = 42,9%, Concepção correta = 42,9%, n = 63

Conceito errôneo: A maioria dos participantes se referiu ao código na questão como sua justificativa, sem nenhum destaque adicional. As palavras mais frequentes incluem "primeiro" e "números", o que demonstra a atenção que esses participantes dedicam ao valor anterior da variável e aos dois números usados no exercício, respectivamente. No entanto, um participante, apesar de ter escolhido a resposta que indicava um conceito errôneo, explicou: "Não tenho certeza, mas se duas [instâncias] da mesma variável forem usadas com números diferentes, se possível, [o resultado] apresentará ambos os números." Caso contrário, seriam 20, pois essa foi a última alteração".

Correto: Uma das palavras mais usadas é "variável". Isso pode indicar que os participantes têm uma compreensão básica do que é uma variável e, portanto, usam o termo com mais frequência.

Além disso, notamos, pelas explicações de alguns participantes, a referência à última alteração feita no valor da variável.

Exemplos disso incluem: "X = 20 vem depois de X = 10 e o último sobrescreverá o anterior". No entanto, um participante demonstra plena consciência desse aspecto das variáveis: "a variável X é alterada para 10, depois para 20, e uma variável só pode ter um valor".

M150: Dificuldades em compreender o efeito das chamadas de função de entrada na execução.

Participantes com a concepção errônea = 39,1%, Concepção correta = 35,7%, n = 115

Conceito errôneo: A palavra "ordem" é a segunda mais mencionada pelos participantes, o que destaca a importância que eles atribuem à execução sequencial que respeita a ordem dos blocos.

Um participante explica, por exemplo: "Esta é a ordem de cima para baixo" e outro diz: "porque [a resposta] está na ordem correta". Além disso, notamos que alguns participantes usam a palavra "resposta", identificando a natureza de perguntas e respostas do programa.

No entanto, eles ainda forneceram a resposta errada devido a uma série de suposições incorretas, como a de que, como não é possível para o participante preencher uma resposta, o computador continuará.

Outra suposição é que a pergunta é dirigida ao computador, portanto o computador responderá e continuará a execução: "Gobo diz que tem ... anos e diretamente diz ...[continua nos próximos blocos]", e outro participante: "Acho que no jogo perguntam a Gobo quantos anos ele tem, então ele ...[continua nos próximos blocos]".

Correto: Observamos novamente duas formas de raciocínio quando os participantes dão a resposta correta. Uma delas tem uma abordagem direta e se baseia na presença da palavra "esperar" tanto na pergunta quanto no texto da resposta, afirmando que a escolha se dá porque "há espera" em ambas as perguntas e respostas. A segunda demonstra que os participantes têm um reconhecimento mais preciso da natureza de pergunta e resposta do programa e, portanto, da necessidade de uma resposta do usuário para que o programa possa continuar. Um participante explica: "se ele te perguntar, então você tem..."

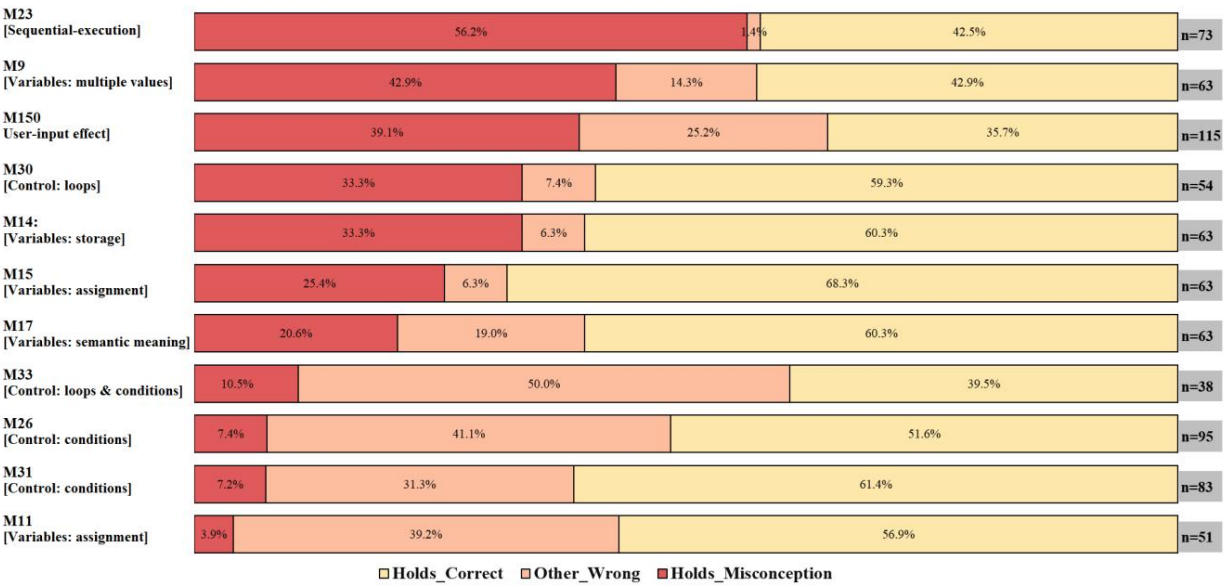


Figura 3: Conceitos errôneos e sua distribuição de respostas, ordenados do mais comum ao menos comum. M14, M26, além dos 3 principais. Os equívocos são discutidos mais a fundo através da análise dos textos abertos dos participantes.

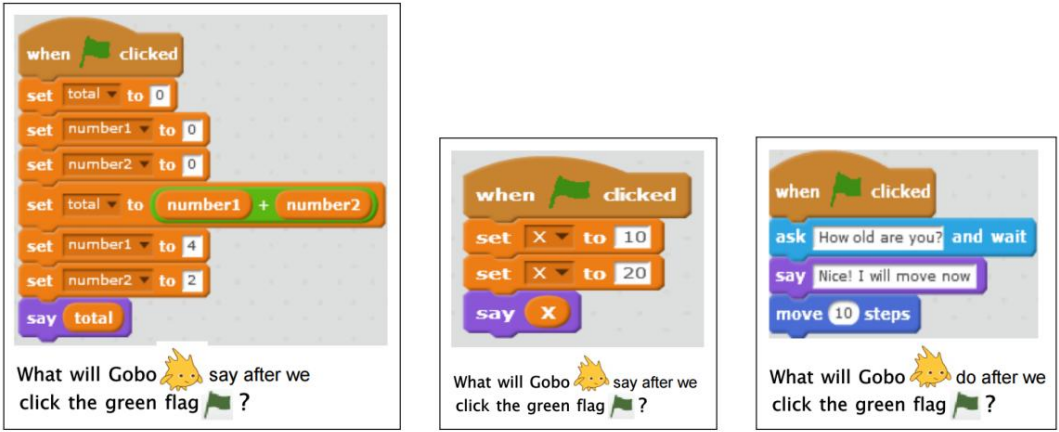


Figura 4: Questões representando M23 (esquerda), M9 (centro) e M150 (direita)

Tabela 2: Palavras mais comuns no texto aberto fornecido pelas crianças para os três principais equívocos.

	M23: execução sequencial			M9: valores múltiplos da variável			M150: efeito de entrada do usuário		
	Classificação 1	Classificação 2	3º lugar	Classificação 1	Classificação 2	3º lugar	Classificação 1	Classificação 2	3º lugar
Mantém a posição correta	antes	valores numéricos		então	definir	variável	espere	responder	até
Contém equívocos	valor	então	adicionar	então	primeiro	números, faz foto	responder	ordem	programa
Outro erro		Não aplicável		então	dizer		o que	saber	faz

para digitar uma resposta, e então ele responderá”, e outro participante diz: “porque você não digitou nada”.

M14: Uma variável é (simplesmente) um emparelhamento de um nome com um Valor variável (com um tipo). Não é armazenado dentro do computador
Participantes com a concepção errônea = 33,3%, Concepção correta = 60,3%, n = 63

Conceito errôneo: No total, 33,3% dos participantes (n=63) escolheram uma opção que indicava um conceito errôneo (ver Figura 3). Especificamente, 7,9% dos participantes escolheram a resposta que indicava que “a variável não está sendo armazenada em nenhum lugar”. Os participantes que apresentaram suas justificativas parecem estar confusos com a opção integrada do Scratch: Variável na nuvem (armazenada no servidor), que foi apresentada como parte da questão. Dois participantes destacaram que sua escolha se deu porque essa opção não estava marcada, que é a configuração padrão do Scratch. 25,4% dos participantes escolheram a resposta que indicava que “a variável é visível apenas na tela”. Para esses participantes, o local onde o armazenamento ocorre é importante e precisa ser detectado.

Um participante diz: “porque você não consegue vê-lo em nenhum outro lugar”, e outro: “não há onde ele estar, então ele simplesmente fica ali”. Outro participante, apesar de ter escolhido a resposta errada, indica uma abordagem analítica que está a um passo da correta. O participante não tem certeza de onde uma variável deve ser armazenada porque o programa não foi executado, dizendo: “o código só atribuirá um valor quando for executado e, como o código ainda não está sendo executado, a variável é irrelevante”.

Correto: Os participantes que responderam corretamente apresentaram raciocínios variados. Alguns deram uma razão concreta, por exemplo, “Fica na memória RAM” e “todos os computadores armazenam dados no hardware para saber o que precisam fazer”. Outros se concentraram na necessidade de salvar todo o programa para que o Scratch “se lembre” dele posteriormente.

M26: Uma condição falsa encerra o programa quando não há ramificação else.

Participantes com a concepção errônea = 7,4%, Concepção correta = 51,6%, n = 95

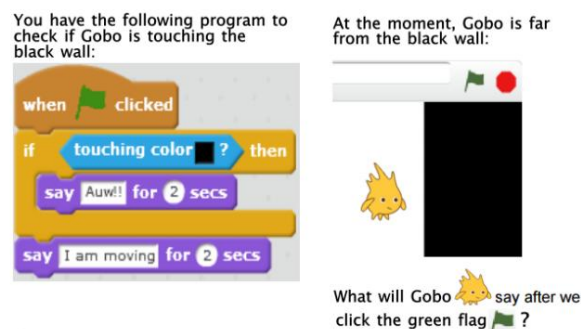


Figura 5: A questão que representa M26

Conceito errôneo: Embora não tenhamos fornecido uma opção de resposta para representar o conceito errôneo (ver Seção 5.4), alguns participantes (7,4%) demonstraram ter esse conceito após analisarem o texto que escreveram. Para esses, o código não é executado, pois a condição é falsa. Por exemplo, um participante afirma: “ele [Gobo] não toca na preta, então nada acontece ali”. Outro participante considera ilógico executar o código: “ele [Gobo] não está tocando na preta, então por que ele executaria os comandos que só se aplicam a ele se ele está tocando na preta?”.

Correto: Os participantes destacam a condição falsa como uma motivação para sua resposta, a partir da perspectiva oposta à dos participantes com a concepção errônea. A condição ser falsa significa que o programa é executado, mas partes dele são ignoradas. Por exemplo, um participante afirma: “ele pula a parte do Auw porque não está em cima da parede preta”, e outro participante concorda: “Gobo diz apenas que estou me movendo porque ele não toca a parede preta”.

4.3 Efeito da idade e do conhecimento prévio de programação [RQ3]

Como a idade e o

conhecimento prévio de programação afetam a manutenção de uma concepção errônea ou a compreensão correta?

4.3.1 Fator idade. Para a análise do efeito da idade, excluímos as faixas etárias de 7 e 17 anos, pois apenas um participante em cada uma delas respondeu ao questionário. Os resultados mostram que existe uma correlação positiva apenas entre a idade e a posse do conceito correto (correlação de Spearman, valor $p = 0,005$). Em outras palavras, quanto mais velha a criança, mais respostas corretas ela obtém. Além disso, ao considerar a categoria da concepção errônea de acordo com a classificação original de Sorva (ver Tabela 1), observa-se uma correlação positiva entre a idade e as respostas corretas às questões sobre concepções errôneas nas categorias “Variável” (correlação de Spearman, valor $p = 0,015$) e “Controle” (correlação de Spearman, valor $p = 0,048$).

Nos diagramas aluviais (ver Figura 6), observamos como as faixas etárias contribuem para as respostas relativas às três principais concepções errôneas. Essa contribuição é representada pela espessura do fluxo da nascente ao destino. Os diagramas mostram que crianças menores de 12 anos têm maior probabilidade de apresentar uma concepção errônea do que crianças mais velhas. No entanto, a relação é significativa apenas para a resposta correta, conforme mencionado anteriormente, e não para a concepção errônea em si.

4.3.2 Conhecimento prévio de programação. O conhecimento relatado de linguagens de programação (Figura 7) mostra que quase dois terços dos participantes já programaram com Scratch, enquanto o terço restante utilizou programação com diversas outras linguagens, como Lego, Alice, Python ou Javascript. Além disso, os participantes relataram onde aprenderam a programar: na escola (62%), em casa (28%) ou em outros lugares, como com amigos, comunidades ou cursos (10%). Como investigamos concepções errôneas sobre o Scratch, exploramos como o conhecimento específico de Scratch se compara ao de outras linguagens de programação em relação à presença de concepções errôneas sobre programação. Os resultados mostram o seguinte: **Conhecer Scratch e outras linguagens:** demonstrou diminuir a tendência a ter concepções errôneas (correlação de Pearson, $r = -0,077$, valor $p = 0,035$) e aumentar a tendência a ter conceitos corretos (correlação de Pearson, $r = 0,151$, valor $p < 0,001$).

Conhecer outros idiomas: demonstrou aumentar a tendência a ter uma concepção errônea (correlação de Pearson, $r=0,071$, valor $p=0,049$).

Conhecer apenas o Scratch: o conhecimento exclusivo do Scratch não se correlaciona com a existência de uma concepção errônea. No entanto, os resultados mostram que ele se correlaciona com respostas incorretas na categoria “Outros_Errados” (correlação de Pearson, $r=0,081$, valor $p=0,025$).

5 DISCUSSÃO

5.1 Observações gerais

Embora alguns possam argumentar que é esperado que crianças mais novas tenham essas ideias equivocadas, suas origens podem ser diferentes.

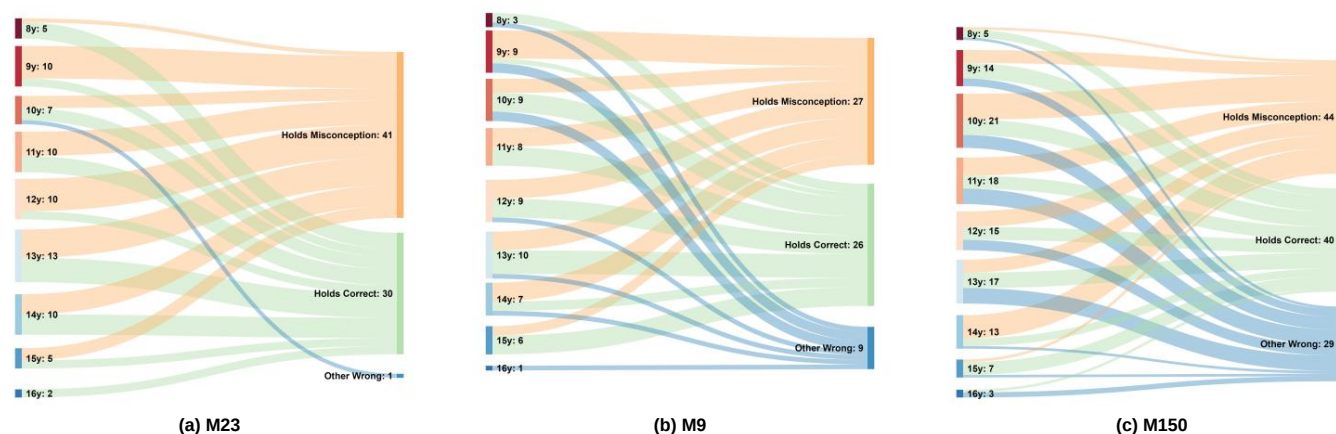


Figura 6: Diagramas aluviais para os três principais equívocos, mostrando o fluxo de respostas das faixas etárias de 8 a 16 anos em direção às categorias de resposta.

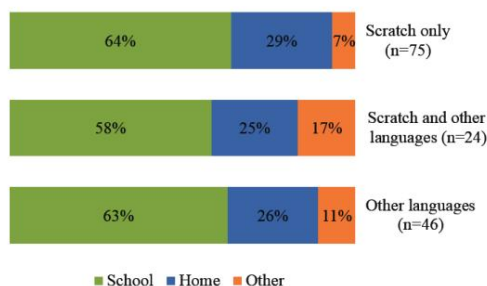


Figura 7: Linguagens de programação relacionadas versus local onde a criança as aprendeu: as escolas são a principal fonte de aprendizado de programação, enquanto quase um terço das crianças indica o lar como local de aprendizado.

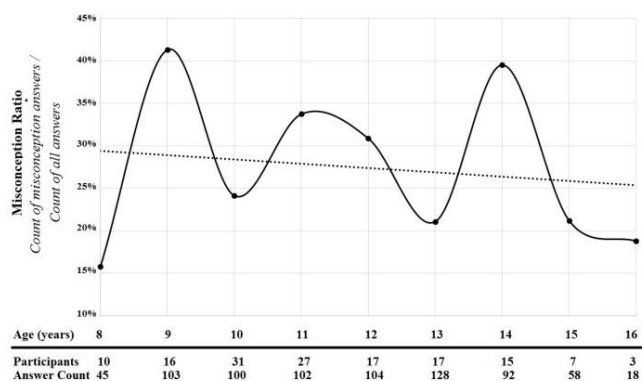


Figura 8: Proporção de respostas incorretas em relação ao total de respostas por idade (ano)

concepções errôneas em estudantes universitários. Em primeiro lugar, a relação com a matemática parece mais disseminada: sempre que uma questão contém números, alguns participantes optam por somar os valores, mesmo quando

Não há nenhuma operação de soma na questão (como em M9). O significado semântico das variáveis é outro aspecto recorrente presente em questões diferentes de M17. Por exemplo, nas concepções errôneas M33 e M31, como usamos o nome da variável "contador", alguns participantes acreditam que o programa deve "contar" a sequência de um até o final, independentemente da condição do laço e da operação dentro dele.

Analisando mais detalhadamente o efeito da idade sobre as concepções errôneas, apresentamos a Figura 8, que mostra a proporção de concepções errôneas entre as crianças por faixa etária. Os resultados mostram um aumento repentino na prevalência de concepções errôneas aos 14 anos. Esse aumento é significativo em comparação com os grupos etários adjacentes (13 e 15 anos). Infelizmente, os dados coletados até o momento não explicam esse aumento e requerem pesquisas adicionais.

5.2 Questões específicas do Scratch A

linguagem de programação tem seu papel em suscitar algumas concepções errôneas. Em pesquisas anteriores, isso contribuiu não apenas para erros sintáticos, mas também para erros relacionados à dificuldade de compreensão do funcionamento conceitual dessa linguagem de programação. Em nosso estudo, notamos que o uso de blocos específicos em nossos problemas de programação gerou comentários comuns e pertinentes por parte dos participantes. Em primeiro lugar, o bloco "Dizer" no Scratch causava confusão sempre que era usado nas perguntas. Apesar de ser um bloco básico e comum, levava a erros quando integrado a uma variável em vez de texto simples.

Muitos participantes neste caso indicaram que o programa dirá o identificador da variável, por exemplo, "X" ou "total", em vez de seu valor. Isso demonstra tanto uma falta de compreensão do conceito de variável quanto o efeito do sentido imperativo linguístico do verbo "Dizer". Uma segunda dificuldade que observamos, relacionada ao Scratch, é o bloco usado para atribuir um valor a uma variável: "Atribuir valor a [variável]".

O uso da preposição "para" gera ambiguidade quanto à direção da tarefa. Por fim, a ordem dos blocos deve ser respeitada, pois, para algumas crianças, os blocos "visualmente conectados" significavam que o programa seria executado de cima para baixo, mesmo na presença de blocos de condição ou repetição.

5.3 Reflexões sobre os resultados e suas implicações

Conceitos errôneos são considerados uma área de dificuldade na aprendizagem de programação. Nossos resultados sugerem que, à medida que nos aproximamos de crianças mais novas, observamos mais dificuldades em responder às perguntas corretamente. Isso se verifica especialmente em relação aos conceitos de variáveis e controle. Esse resultado confirma pesquisas anteriores que demonstraram que crianças menores de 11 anos têm dificuldades em compreender esses conceitos, além dos conceitos de paralelização e procedimentos [9, 16]. Nossos resultados mostram, no entanto, algumas observações diferentes daquelas feitas com estudantes universitários: i) as crianças tendem a realizar operações matemáticas (principalmente somatórios) sempre que números estão presentes nos exercícios, e ii) tendem a fazer suposições com base em sua compreensão dos nomes semânticos das variáveis. Consequentemente, acreditamos que exercícios mais diversificados devem ser desenvolvidos para incluir operações com strings e booleanos, além da seleção cuidadosa dos nomes dos identificadores nesses exercícios.

Além disso, nossos resultados sugerem que as crianças que indicaram programação apenas no Scratch têm dificuldades em compreender os conceitos corretamente e tendem a escolher outras respostas incorretas. Esse resultado destaca duas questões. Primeiro, educadores e pesquisadores são incentivados a identificar e compreender novas concepções errôneas induzidas pelo Scratch como linguagem. Apresentamos algumas observações a partir de nosso conjunto de dados (ver Seção 5.2), mas mais pesquisas ainda são necessárias. Em segundo lugar, nossos resultados sugerem que crianças mais novas começam a aprender programação em Scratch (a idade correlaciona-se positivamente com o número de linguagens de programação relatadas, além do Scratch), e fazem isso principalmente nas escolas. Esses usuários mais jovens do Scratch parecem ter mais dificuldades para compreender corretamente os conceitos em nosso estudo. Esse resultado confirma pesquisas anteriores que constataram que projetos em Scratch apresentam uma baixa porcentagem de construções conceituais, como variáveis, procedimentos e declarações condicionais [1, 16, 23]. Consequentemente, acreditamos que os professores do ensino fundamental devem desenvolver materiais mais complexos e ricos em conceitos para o Scratch.

Por fim, alguns participantes demonstraram sinais de dificuldade em conciliar pensamentos contraditórios ao fundamentarem suas respostas (ver Seção 4.2). Isso indica nossa crença de que ter uma concepção errônea não é uma questão binária: ou você entende o conceito ou não. Pelo contrário, pode ser um passo para a compreensão completa do conceito e, quando a confusão é identificada, torna-se mais fácil para os educadores fornecerem a informação que falta.

5.4 Ameaças à validade

Como

todos os estudos, este artigo apresenta algumas limitações. Uma ameaça externa à validade provém da experiência em programação relatada pelos participantes. No questionário, incluímos cinco perguntas sobre a experiência prévia em programação, incluindo questões para identificar o conhecimento de conceitos específicos. Os participantes ainda podem ter avaliado mal sua própria experiência e fornecido indicações falsas. Além disso, uma ameaça à validade decorre do uso de questões de múltipla escolha, pois algumas crianças podem ter chutado a resposta mesmo sem o conhecimento necessário. Minimizamos essas duas ameaças adicionando um texto aberto após cada questão de múltipla escolha e, em seguida, filtrando rigorosamente qualquer resposta para a qual o participante tenha indicado, no texto aberto, que chutou a resposta ou que não a compreendeu.

A questão ou o conhecimento para respondê-la. Uma ameaça interna à validade é a própria concepção da questão e das possíveis respostas para M26. As respostas que fornecemos não incluíram uma resposta que refletisse a ideia errada: neste caso, a de que o programa não faria nada. Apesar desse problema de design, 7,2% dos 95 participantes que responderam a essa pergunta apresentaram a ideia errada, que se baseava no texto que forneceram usando a opção de resposta "Outro".

6 CONCLUSÕES

Nosso artigo visa explorar as concepções errôneas sobre programação apresentadas por alunos em idade escolar. O estudo baseia-se em um questionário de múltipla escolha com exercícios de programação em Scratch. Participaram do estudo 145 crianças, com idades entre 7 e 17 anos e alguma experiência prévia em programação. Os resultados mostram que os alunos mais jovens, em idade escolar, de fato apresentam concepções errôneas que os levaram a cometer erros ao desenvolver um pequeno script em Scratch. As três principais concepções errôneas abrangem múltiplos conceitos: M23: a dificuldade em compreender a sequencialidade das instruções; M9: a dificuldade em compreender que uma variável armazena apenas um valor por vez; e M150: a dificuldade em compreender a natureza interativa de um programa quando é necessária a entrada do usuário. As origens dessas concepções errôneas são variadas e incluem a grande influência de números e operações matemáticas na mentalidade das crianças, a influência do significado semântico do identificador da variável e a expectativa equivocada sobre o que um computador pode fazer, ou seja, a incompreensão do conceito de máquina. Ao analisar o efeito da idade, descobrimos que as crianças mais velhas tendem a responder aos exercícios corretamente. Além disso, os resultados mostram que o conhecimento de Scratch, além de pelo menos outra linguagem de programação, influencia positivamente a escolha da resposta correta. Já as crianças que relataram programar apenas em Scratch apresentaram maior tendência a escolher outras respostas incorretas. Por fim, os exemplos observados no experimento indicam que ter uma concepção errônea é, de fato, um passo em direção à compreensão correta e completa do conceito. Nossos resultados sugerem que educadores e professores devem considerar o efeito das concepções errôneas o mais cedo possível. Além disso, devemos estar atentos às novas concepções errôneas induzidas pelo Scratch como linguagem, devido, entre outros motivos, ao uso de blocos visualmente conectados e ao uso de palavras-chave específicas em seu conjunto de blocos. Finalmente, o material e as aulas de Scratch devem integrar mais exercícios ricos em conceitos que destaquem áreas como variáveis e controle de execução. Isso é especialmente necessário para crianças que começam a aprender programação em Scratch. Em trabalhos futuros, temos duas direções principais. Primeiro, pretendemos explorar e identificar concepções errôneas específicas do Scratch. Segundo, pretendemos estudar mais a fundo o efeito do aprendizado de outra linguagem de programação em aprendizes de Scratch. Finalmente, pretendemos desenvolver e testar materiais e métodos de ensino que tenham menor probabilidade de induzir concepções errôneas em crianças.

RECONHECIMENTO

Gostaríamos de agradecer à equipe do programa Science Live do NEMO e a todos os funcionários pelo apoio. Agradecemos também aos colegas e estudantes voluntários que participaram da realização do experimento no NEMO museu.

REFERÊNCIAS

- [1] Elthimia Aivaloglou e Feliene Hermans. 2016. Como as crianças programam e como sabemos. Anais da Conferência ACM de 2016 sobre Pesquisa Internacional em Educação Computacional - ICER '16 (2016). DOI:http://dx.doi.org/10.1145/2960310.2960325
- [2] Erik Barendsen, NataiĖja Grgurina e Jos Tolboom. 2016. Um novo currículo de informática para o ensino secundário nos Países Baixos. Informática nas escolas: melhoria do conhecimento e da percepção da informática (2016), 105–117. DOI: http://dx.doi.org/10.1007/978-3-319-46747-4_9
- [3] A. Berglund e R. Lister. 2010. Programação introdutória e o triângulo didático. In Anais da 12ª Conferência Australiana sobre Educação em Computação. 35–44. http://dl.acm.org/citation.cfm?id=1862219.1862227 [4] B. Du Boulay. 1986. Algumas dificuldades de aprender a programar. Journal of Educational Computing Research 2, 1 (1986), 57–73. DOI:http://dx.doi.org/10.2190/3LFX-9RRF-67T8-UVK9 arXiv:https://doi.org/10.2190/3LFX-9RRF-67T8-UVK9 [5] S. Dasgupta e B. Hill. 2017. Aprendendo a programar em linguagens de programação localizadas. Anais da 4ª Conferência ACM sobre Aprendizagem em Escala (2017). DOI:http://dx.doi.org/10.1145/3051457.3051464
- [6] D. Doukakis, M. Grigoriadou e G. Tsaganou. 2007. Compreendendo o conceito de variável de programação com analogias interativas animadas. Anais da 8ª Pesquisa Helênica Europeia em Matemática Computacional e suas Aplicações (2007).
- [7] P. Fung, M. Brayshaw, B. Du Boulay e M. Elsom-Cook. 1990. Rumo a uma taxonomia de concepções errôneas de iniciantes sobre o interpretador Prolog. Instructional Science 19, 4-5 (1990), 311–336. DOI:http://dx.doi.org/10.1007/bf00116443 [8] K. Goldman, P. Gross, C. Heeren, G. Herman, L. Kaczmarczyk, MC Loui e C. Zilles. 2008. Identificando conceitos importantes e difíceis em cursos introdutórios de computação usando um processo Delphi. ACM SIGCSE Bulletin 40, 1 (2008), 256. DOI:http://dx.doi.org/10.1145/1352322.1352226
- [9] F. Hermans e E. Aivaloglou. 2017. Ensinando princípios de engenharia de software para alunos do ensino fundamental e médio: um MOOC no Scratch. Em Anais da 39ª Conferência Internacional de Engenharia de Software: Trilha de Engenharia de Software e Educação. 13–22. DOI:http://dx.doi.org/10.1109/ICSE-SEET.2017.13 [10]
- Hai Hong, Jennifer Wang e Sepehr Hejazi Moghadam. 2016. Educação em Ciência da Computação K-12 nos EUA: Informática nas Escolas: Aprimoramento do Conhecimento e da Percepção da Informática (2016), 142–154. DOI:http://dx.doi.org/10.1007/978-3-319-46747-4_12
- [11] E. Kurvinen, N. Hellgren, E. Kaila, M. Laakso e T. Salakoski. 2016. Conceitos errôneos de programação em uma prova de curso introdutório de programação. Anais da Conferência ACM sobre Inovação e Tecnologia no Ensino de Ciência da Computação (2016). DOI:http://dx.doi.org/10.1145/2899415.2899447
- [12] L. Ma. 2007. Investigando e aprimorando os modelos mentais de conceitos de programação de programadores iniciantes. Tese de doutorado. Universidade de Strathclyde, Reino Unido. http://ethos.bl.uk/OrderDetails.do?uin=uk.bl.ethos.444415
- [13] L. Ma, J. Ferguson, M. Roper e M. Wood. 2011. Investigando e aprimorando os modelos de conceitos de programação mantidos por programadores novatos. Computer Science Education 21, 1 (2011), 57–80. DOI:http://dx.doi.org/10.1080/08993408.2011.554722
- [14] Roy D. Pea. 1986. "Bugs" conceituais independentes de linguagem em programação para iniciantes. Journal of Educational Computing Research 2, 1 (1986), 25–36. DOI: http://dx.doi.org/10.2190/689t-1r2a-x4w4-29j2
- [15] R. Putnam, D. Sleeman, J. Baxter e L. Kuspa. 1986. Um resumo das concepções errôneas de programadores de BASIC do ensino médio. Journal of Educational Computing Research 2, 4 (1986), 459–472. DOI:http://dx.doi.org/10.2190/fgn9-dj2f-86v8-3fau [16] Linda Seiter e Brendan Foreman. 2013. Modelagem das progressões de aprendizagem do pensamento computacional de alunos do ensino fundamental. Anais da nona conferência internacional anual da ACM sobre pesquisa em educação computacional - ICER '13 (2013). DOI:http://dx.doi.org/10.1145/2493394.2493403 [17] Simon. 2011. Atribuição e sequência. Anais da 11ª Conferência Internacional sobre Pesquisa em Educação Computacional (2011). DOI:http://dx.doi.org/10.1145/2094131.2094134 [18] Simon e S. Snowdon. 2011. Explicando o código de programa. Anais do 7º Workshop Internacional sobre Pesquisa em Educação Computacional (2011). DOI:http://dx.doi.org/10.1145/2016911.2016931 [19] D. Sleeman, R. Putnam, J. Baxter e L. Kuspa. 1986. Pascal e alunos do ensino médio: um estudo de erros. Journal of Educational Computing Research 2, 1 (1986), 5–23. DOI:http://dx.doi.org/10.2190/2xpp-ltyh-98nq-bu77 [20] J. Sorva. 2008. Compreensões iguais, mas diferentes, de alunos sobre variáveis primitivas e de objeto. Anais da 8ª Conferência Internacional sobre Pesquisa em Educação Computacional (2008). DOI:http://dx.doi.org/10.1145/1595356.1595360 [21] J. Sorva. 2012. Simulação visual de programas no ensino introdutório de programação. Tese de doutorado, Universidade Aalto. (2012). http://urn.fi/URN:ISBN:978-952-60-4626-6
- [22] Juha Sorva. 2013. Máquinas nomenclais e educação introdutória em programação. ACM Transactions on Computing Education 13, 2 (2013), 1–31. DOI:http://dx.doi.org/10.1145/2483710.2483713 [23]
- Alaaeddin Swidan, Alexander Serebrenik e Feliene Hermans. 2017. Como os programadores do Scratch nomeiam variáveis e procedimentos? 2017 IEEE 17th International Working Conference on Source Code Analysis and Manipulation (SCAM) (2017). DOI:http://dx.doi.org/10.1109/scam.2017.12 [24]
- D. Teague, M. Corney, A. Ahadi e R. Lister. 2013. Um estudo qualitativo de pensamento em voz alta dos estágios neopiagetianos iniciais de raciocínio em programadores novatos. Em Anais da 15ª Conferência Australiana de Educação em Computação. 87–95. http://dl.acm.org/citation.cfm?id=2667199.2667209